

Computational Identity and Computational Functionalism

A problem-driven conceptual guide

From thought experiments to a disciplined theory architecture

August 2026

What this note does

This note has two deliberately separated parts.

Part I develops a theory of computational identity from a series of diagnostic failure cases. Arbitrary state relabeling, function-versus-algorithm, local abstraction, non-linear granularity, the stone-factorization challenge, ambiguity about causal participation, and the split between actual and counterfactual organization jointly motivate a picture in which physical episodes support structured spectra of admissible computational abstractions grounded in coherent value representation and actual computational provenance.

Part II applies that computational ontology to computational functionalism. It formulates constraints on the *form* of a consciousness-making computational property: its required invariances, the proper role of counterfactual structure, and the priority of the process actually instantiated over unrealized alternatives.

1 The problems that force the theory

Thought experiment 1: The relabeling trick

Setup. Imagine that a computation is represented only by a finite sequence of internal states: first state *A*, then *B*, then *C*, and so on. Now take any physical object whose global microstate changes over time – a stone, a turbulent fluid, a processor, a brain. Record a sequence of distinct snapshots.

A dictionary can now be invented after the fact: call the first physical snapshot “computational state *A*,” the second “state *B*,” the third “state *C*,” and so forth.

What goes wrong. If a successful computational implementation requires only that a sequence of physical states can be put in correspondence with a sequence of abstract states, almost any sufficiently rich history can be made to implement almost any finite run. The alleged computation is doing no explanatory work. The dictionary is doing all of it.

What the example forces us to add. A realized computation cannot be represented as a bare sequence of atomic global states. The representation must preserve some of the internal organization of the actual process: which events occurred, which value-bearing tokens were present, where those tokens were routed, which events consumed them, what new tokens were produced, and which parts of the process were independent.

There is a second constraint. The interpretation of values cannot be reinvented at every instant. If one physical carrier is read as the abstract value 7 here and the same type of carrier is read as 23 later merely because that makes the desired trajectory fit, the relabeling trick has returned in local form. What is required is not one unique physical code for the entire system, but a **coherent representation scheme**: a stable family of local or typed decoding rules that can be reused across the episode and does not depend on knowing the completed trajectory. The rule may itself depend on an explicitly represented mode or local context; what is excluded is an ad hoc change of interpretation indexed only by where one happens to be in the desired history.

Concept forced by the example

The first primitive object should be an **actual valued computational provenance structure**, not a state sequence. A run has internal organization, and its value interpretation must be trajectory-independent and reusable. The order “first, second, third” is only one projection of that richer structure.

Thought experiment 2: The same function, two different computations

Setup. Consider the function that maps an integer N to the sum of the integers from 1 to N . One routine initializes an accumulator and adds 1, then 2, then 3, and so on. Another evaluates the closed-form expression $N(N + 1)/2$.

What goes wrong. Both routines compute the same input-output function. Yet they plainly do not instantiate the same internal computation. They generate different intermediate values, different dependency graphs, different timing behavior, different possibilities for interruption, and different suboperations.

So “same function” is too coarse to mean “same computation.”

What the example forces us to add. Distinguish the **extensional function** from the **algorithmic or causal organization** that realizes it. Function identity is one possible abstraction of computation, usually a very coarse one.

Concept forced by the example

A function forgets the path. A computation, at a finer description, includes the structured production of the result. Therefore function identity and computational identity must not be primitive synonyms.

Thought experiment 3: The subroutine disappears inside a larger program

Setup. Put those two summation routines inside two larger programs that are otherwise identical. Suppose the surrounding code only supplies N and receives the returned sum. It cannot inspect intermediate accumulator values, pause the routine halfway through, or observe its internal memory.

At a fine description, the two programs differ because one contains a loop and the other a closed-form computation. At the interface seen by the surrounding program, the two subroutines are interchangeable.

What goes wrong. We now have two apparently conflicting statements:

- the programs contain different internal computations;
- the programs can be treated as the same larger computation once the summation component is viewed only through its interface.

Neither statement should be discarded.

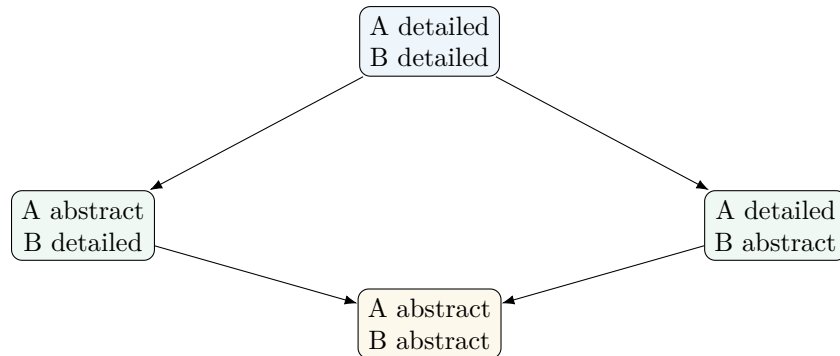
What the example forces us to add. Computational identity is always identity *at an abstraction*. A valid abstraction may collapse a whole subcomputation into a single component when the distinctions inside that component make no difference to the surrounding structure being retained.

Concept forced by the example

Abstraction is not denial of the finer computation. It is a quotient of it: several distinct fine structures may occupy the same abstract computational role.

Thought experiment 4: Granularity is not a slider

Setup. Now imagine a program containing two complicated routines, A and B . We can retain the internals of both, hide the internals of A while retaining B , hide B while retaining A , or hide both.



What goes wrong. The two middle descriptions are incomparable. Neither is simply “more detailed” than the other. One keeps information about A that the other discards; the other keeps information about B that the first discards.

Therefore a global scalar called “the level of granularity” is misleading.

What the example forces us to add. Computational abstractions form a **partially ordered family** under refinement. In sufficiently regular cases this family can have lattice-like structure: common refinements, common coarse-grainings, and many incomparable branches.

Concept forced by the example

Granularity is closer to a structured vector than to a scalar. More precisely, **abstraction is structured information loss**: what matters is not how much detail is discarded, but which distinctions are discarded.

Thought experiment 5: The stone that allegedly factors a number

Setup. Hand someone a huge semiprime and a stone. Suppose they claim that, with a sufficiently ingenious interpretation of the stone’s changing microstate, the stone is performing the factorization.

The challenge is simple: identify where the input is represented, where the intermediate factors or arithmetic quantities are represented, how later physical events actually depend on earlier carriers, and where the output can be read without placing the factorization itself inside the decoding dictionary.

What goes wrong. A global state-labeling map can always hide arbitrary work in the interpretation. But that does not show that the relevant arithmetic organization occurred inside the physical process.

What the example forces us to add. The actual-run notion must preserve **locality, provenance, value flow, component reuse, and causal decomposition**. It must also

use a coherent representation scheme for values. An admissible computational interpretation cannot obtain its structure solely by giving each whole-system snapshot an arbitrary name or by changing the decoding rule opportunistically from one event to the next.

What not to conclude

This does not prove that a stone realizes no computations. A sufficiently rich physical episode may genuinely contain computational patterns. The claim is narrower: if a computation is present, it must be witnessed by structure in the episode itself rather than by retrospective global relabeling.

Thought experiment 6: Participation, contribution, and value support

Setup. Consider a three-input OR component. In one actual occurrence it receives

`true, true, false` $\rightarrow$ `true`.

All three input signals physically arrive at the component. Should the occurrent computational structure contain all three inputs? Yes, if the relation means that a value token was actually supplied as an operand. But that is not yet the same as saying that each token was important to the particular output value.

There is a useful stronger notion. Relative to the operation OR and to this actual input tuple, a **value support** for the observed output is a subset of the actual input-value assignments that is sufficient to fix that output while the remaining inputs are left unspecified. Here either `input 1 = true` alone or `input 2 = true` alone suffices to guarantee the output `true`; `input 3 = false` does not. The two true inputs therefore form two alternative minimal supports.

For an AND gate receiving `true, true`, by contrast, the observed output `true` has one joint minimal support containing both true inputs. For an XOR gate, fixing the parity result generally requires a different support pattern again.

What goes wrong. A single edge relation cannot carry all of this information. At least four questions have been mixed together:

- Which tokens actually arrived and participated in the event?
- Which actual-value subsets are sufficient to support the particular output value?
- Which particular values count as causes or contributors under some theory of actual causation?
- Which input positions can affect the operation across alternative cases?

The first question is directly occurrent. The fourth is fully modal. The second is anchored to an actual occurrence but uses the local operation semantics to quantify over alternatives for the unspecified inputs; it is therefore **occurrence-indexed but locally modal**. The third is a further explanatory notion and is delicate under redundancy and overdetermination.

What the example forces us to add. The base occurrent object should record **provenance and participation**. It may be enriched by a **value-support hypergraph**: for each actually produced value, record its minimal sufficient sets of actual value tokens relative to the local operation semantics. Actual contribution and full modal dependence should remain separate notions.

Concept forced by the example

Keep four structures distinct:

Participation / provenance: this token was actually transported to, consumed by, or produced by this event.

Value support: this set of actual value assignments is sufficient to fix this actual result, relative to the local operation semantics.

Actual contribution: this actual value counts as a cause or contributor to this result under an explicit contribution criterion.

Modal dependence: this variable or port can affect the operation across admissible alternative cases.

What not to conclude

A value-support structure is not purely non-counterfactual. It is indexed by an actual occurrence, but sufficiency is assessed using the local rule of the operation. This makes support useful for analyzing how a result is organized without pretending that it belongs to the same ontological layer as raw provenance.

Thought experiment 7: One actual run does not identify the whole mechanism

Setup. Observe a component three times. On the three input cases that actually occur, its outputs agree with an AND gate. The fourth input case never occurs.

Or observe a brain for one minute in perfect detail. Suppose we know every actual computational dependency that occurred during that minute, but the subject never encountered millions of other possible sensory inputs.

What goes wrong. From the actual episode alone, we cannot infer every unrealized transition. Many distinct mechanisms may agree perfectly on everything that occurred and diverge only on cases that did not occur.

If we insist that every statement about computation must already include all these unrealized cases, then the computational identity of an actual episode becomes immediately counterfactual. If we refuse all counterfactual structure, we should not pretend that the actual run uniquely determines the full mechanism.

What the example forces us to add. Split the theory into two legitimate notions.

Concept forced by the example

Occurrent computation asks what computational organization was actually instantiated in this episode.

Modal computation asks what computational mechanism the system implements across the relevant alternative inputs and interventions.

The occurrent notion deliberately concludes less. It is stronger than a state trajectory because it contains actual represented provenance and composition, but weaker than a complete modal mechanism because it leaves unvisited cases open.

Thought experiment 8: Two different computers running “the same thing”

Setup. Take two old PCs from different manufacturers. Their transistor layouts differ, their low-level timings differ, perhaps parts of their architecture differ, yet both run Windows 95 and behave similarly at familiar software interfaces.

What goes wrong. It would be wrong to say that their entire computational organization is identical. It would also be wrong to say that they share nothing computationally simply because their hardware is different.

Moreover, “Windows 95” itself is not a mathematically complete specification of one abstract computation. It is also the name of a historical software artifact, with versions, binaries, provenance, and conventions about what counts as the same software.

What the example forces us to add. Two systems can share a large **common region of computational structure** without having identical spectra. Ordinary names such as “the same program” may designate some part of that common structure, but the artifact-identity question is not identical to the computational-identity question.

Concept forced by the example

The useful primitive comparison is not merely “same computation: yes or no.” It is: **what computational structure do the two systems share, and how does that shared structure sit inside each spectrum?**

Thought experiment 9: The giant lookup-table replica

Setup. Model an open synchronous system by a global transition rule: current internal state plus current input determines the next internal state and current output. Now imagine two realizations of exactly the same global rule.

One is a richly factored mechanism with many reusable local computations. The other is an enormous lookup table with one stored answer for every possible state-input pair.

What goes wrong. At the level of the global transition function, they are identical. Yet their fine internal spectra may overlap very little. The table preserves the global transduction while destroying most of the original factorization into local subcomputations.

This shows again that global functional equivalence is much weaker than fine computational equivalence.

What the example forces us to add. Resource constraints create a new question. A huge table can avoid exploiting internal structure by paying with memory. As available resources are restricted, some factorizations may become unavoidable – although compression alone does not guarantee recovery of the original factorization.

Concept forced by the example

This suggests a third, optional axis of the theory: **resource-constrained computational rigidity**. Among all realizations of the same abstract computation that satisfy a resource budget, which computational substructures occur in every realization?

2 Assembling the model

The thought experiments above are not independent curiosities. They force a compact architecture. The theory needs only a few kinds of objects, but it must keep them separate.

2.1 Physical episodes and computational provenance

A **physical episode** is a bounded piece of the actual evolution of a physical system. For computational purposes, the episode should not be reduced to a sequence of global snapshots. What matters

is its internal actual organization: events, value-bearing carriers or tokens, transport, consumption, production, routing, control, reuse, and partial order.

Call this enriched object the episode’s **actual valued computational provenance structure**. The word “provenance” is deliberate. The base object records how pieces of represented information actually moved through and were transformed by the episode. It does not silently identify that factual participation relation with a stronger theory of which values were the decisive causes of each result.

Values require a representation discipline. The theory should not require one unique encoding for the entire physical system: the same abstract Boolean may be electrical in one subsystem, optical in another, and magnetic in a third. What it requires is a **coherent representation scheme**: a stable family of local or typed decoding rules, together with explicit translations where necessary. The scheme must be reusable across occurrences and cannot be selected separately for each point after inspecting the completed history. Context-dependent encodings are allowed when the context and translation rule are themselves part of the represented organization.

This requirement blocks a local version of the relabeling trick. A physical difference counts as a represented computational value only through a representation rule that has some stable domain of application; the decoder cannot smuggle the desired computation into a trajectory-indexed dictionary.

Repeated uses of the same component can reveal more structure without introducing global counterfactual commitments. If the component is actually observed on several input tuples, the episode may support a **partial operation table**: what that component in fact did on the cases that occurred. Unvisited input tuples remain unspecified at the occurrent level.

A second enrichment is possible when the local operation type is independently available. For one actual operation occurrence, define a **value support** as a set of actual input-value assignments sufficient to fix the actual output value while the remaining inputs are left free. Minimal supports form a hypergraph over the actual value tokens. This object is useful but must be classified carefully: it is anchored to an actual occurrence yet uses local modal semantics to test sufficiency. It is therefore richer than provenance and weaker than importing the modal organization of the whole system.

Working definition

An **occurrent computational abstraction** is a structure-preserving compression of an actual valued computational provenance structure under a coherent representation scheme. It may hide internal details, merge events, or replace a detailed subsystem by an abstract component, but only when the retained value flow, participation structure, ordering, and composition are genuinely supported by what occurred.

2.2 Four relations that should not be collapsed

The phrase “causal structure” is too overloaded to serve as a single primitive. At least four relations should be kept conceptually separate.

Relation	What it says	Modal status
Participation / provenance	A token was actually delivered to, consumed by, transported through, or produced by an event	Strictly occurrent base relation
Value support	A set of actual input-value assignments is sufficient for the actual result relative to the local operation semantics	Occurrence-indexed, locally modal
Actual contribution	A particular actual value contributed to a particular actual result under an explicit theory of actual causation or contribution	Optional explanatory enrichment
Modal dependence	An input, variable, or component can affect another across admissible alternative cases	Fully modal mechanism relation

The distinctions matter even for tiny examples. In `OR(true,true,false)=true`, all three tokens participate, the false token belongs to no minimal sufficient support for the true output, and the two true tokens appear as alternative minimal supports. Yet all three ports belong to the gate's modal dependence structure. A theory of actual causation may classify the two redundant true values in still another way.

The support relation is naturally hypergraphic rather than merely graphical. An AND output may require a joint set of values; an OR output may have several alternative singleton supports. Keeping this structure separate prevents the theory from pretending that one binary edge relation captures participation, sufficiency, explanatory contribution, and counterfactual dependence at once.

2.3 Modal mechanisms

A **modal computational mechanism** enriches the actual organization with stable rules across a relevant family of possible inputs and interventions. It says not only how one episode unfolded, but how the same organization would transform other admissible states or respond to perturbations.

Modal structure is what distinguishes an actual partial regularity from a full implemented rule. The component observed on three AND-compatible cases may support an occurrent partial operation; calling it an AND gate in the full mechanistic sense requires the missing modal commitment.

Working definition

A **modal computational abstraction** is an admissible abstraction of a physical mechanism that preserves the computational organization across the relevant alternatives, not merely along one realized history.

2.4 The computational spectrum

A system should not be assigned one privileged computational level. It supports many admissible abstractions, often incomparable because different subsystems can be abstracted independently.

The structured family of these abstractions is its **computational spectrum**.

For one actual episode we obtain an **occurrent computational spectrum**. For the underlying mechanism we obtain a **modal computational spectrum**.

Schematic notation is useful even in an informal account:

$$\text{Spec}_{\text{occ}}(E), \quad \text{Spec}_{\text{mod}}(M).$$

These are not merely unstructured bags of descriptions. They carry at least a refinement relation: one member refines another when it preserves every distinction retained by the coarser member and adds further structure.

Implementation without an observer-selected level

A physical system implements an abstract computation C when C occurs as an admissible member of the relevant computational spectrum.

The observer may decide to ask about C , but does not create C by selecting a level. The system either contains the required computational structure or it does not.

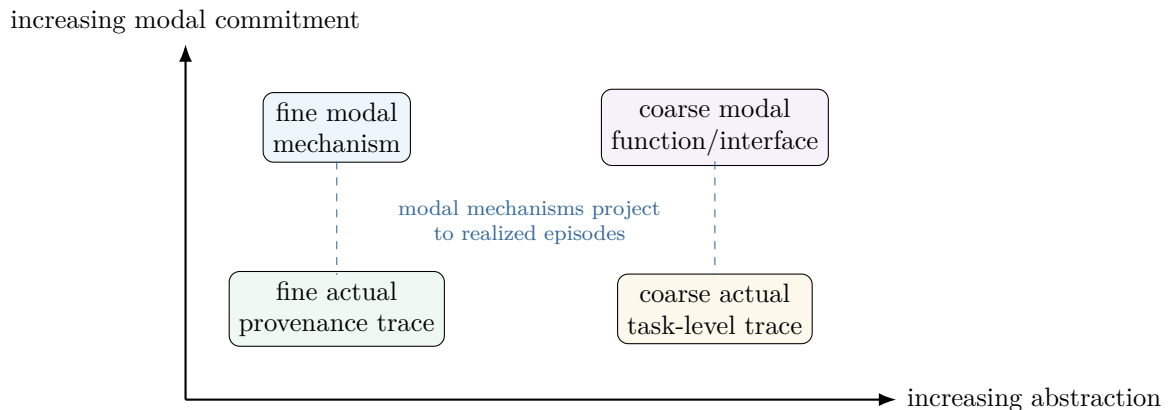
2.5 Why there are two spectra

The occurrent and modal spectra answer different questions.

	Occurrent spectrum	Modal spectrum
Target	One actual episode	The stable mechanism
Contains	Actual events, represented values, participation/provenance, partial operations, local composition; optionally value supports when local operation semantics are supplied	Transition organization and modal dependence across relevant inputs and interventions
Strength	Weaker	Stronger
Leaves open	Unvisited cases	Nothing inside the specified modal domain
Blocks	Mere relabeling of snapshots	Accidental agreement on one history

A modal realization, when actually executed, should produce a corresponding occurrent realization under the same representation scheme. But a single occurrent realization generally admits many possible modal completions. The occurrent trace can constrain those completions by accumulating partial operation tables without selecting values for cases that never occurred.

This relationship can be pictured without implying that abstraction and modality are the same dimension:



3 The internal geometry of a computational spectrum

Once the spectrum replaces the picture of one privileged level, several comparative notions become natural.

3.1 Refinement

A computation A **refines** a computation B when A retains all the distinctions of B and additional ones. Replacing a loop by a black-box “SUM” component moves from a finer member of the spectrum to a coarser one. Abstracting a different component gives a different branch, not necessarily a higher or lower point on one scalar ladder.

3.2 Containment

A system X has **computational containment** over Y when the relevant computational structure of Y embeds into the spectrum of X . Schematically one may write

$$\text{Spec}(Y) \hookrightarrow \text{Spec}(X),$$

with the understanding that the embedding should respect refinement relations, not merely set membership.

Emulation is a natural source of containment: a richer system may realize the computational organization of another system while also realizing additional structures.

3.3 Computational equivalence

Two systems are **spectrally equivalent** in a strong sense when their relevant computational spectra have the same structure. This is much stronger than saying that they compute the same input-output function or run the same software artifact.

Strong spectral equivalence is therefore not the default notion of “same computation.” It is one precise endpoint in a family of comparisons.

3.4 Common computational core

The most useful comparison between different systems is often their **common computational core**: the structured family of computational abstractions realized by both.

For systems X and Y , the informal idea is the structured overlap

$$\text{Core}(X, Y) \sim \text{Spec}(X) \cap \text{Spec}(Y).$$

The notation hides technical choices about isomorphism and embedding, but the concept is clear: two machines can be deeply different while still sharing a rich common computational organization.

Takeaway

The question “are these two machines doing the same computation?” is usually underspecified. Better questions are:

- Which computations occur in both spectra?
- Does one spectrum contain a faithful copy of part of the other?
- What is the richest common abstraction?

- At which refinements do the systems diverge?

4 Objectivity, context, and naming

The theory separates three issues that are often collapsed.

First: physical realization. Whether an admissible computational structure occurs in the spectrum is intended to be an objective fact about the system. Admissibility includes coherent representation: a computational value cannot be rescued by changing the codebook opportunistically from one occurrence to the next.

Second: abstraction. The spectrum contains many abstractions at once. No single member has to be privileged as “the” computational level.

Third: reference. Human language may be vague about which abstract object is intended. “Windows 95,” “the same algorithm,” or “the same agent” can underdetermine what part of the spectrum is being named.

This locates conventionality in the question rather than in the physical realization. A name may fail to specify its target precisely; that does not imply that membership in the spectrum is observer-relative.

Context also matters locally. Two subroutines can be interchangeable for a caller that sees only input and output, yet distinguishable for a caller that observes timing, intermediate memory, power use, or interruptibility. The admissible abstraction must therefore preserve every distinction that remains accessible through the interface of the larger computation.

5 Consequences that fall out of the model

The model is informal here, but several theorem-like consequences follow almost immediately from the architecture.

1. Bare trajectory identity is too weak

If computational identity preserves only temporal succession of atomic global states, trivial relabelings are unavoidable. Any nontrivial occurrent notion must preserve more internal structure than a state sequence.

2. Value identity requires a coherent representation scheme

An occurrent implementation cannot be certified by a decoder that changes opportunistically along the trajectory. Representation may be heterogeneous across physical subsystems, but each local or typed code must be stable enough to be reused across occurrences without consulting the completed run.

3. Participation, support, contribution, and modal dependence are different

A value can be an actual operand without belonging to any minimal support of the observed result; a minimal support can be one among several redundant alternatives; an actual-causation criterion can classify those alternatives differently; and a port can belong to the modal dependence structure even when its current token is support-irrelevant. These relations should not be collapsed into one causal edge.

4. Value support is intermediate rather than purely occurrent

Minimal sufficient supports are indexed by an actual event and actual values, but their sufficiency is evaluated using the local operation semantics over unfilled inputs. They are therefore a principled intermediate object between raw provenance and full-system modal structure.

5. Function identity is a quotient, not full identity

Distinct computations can collapse to the same input-output function. Therefore equality of functions can be recovered as a coarse spectral equivalence without being mistaken for equality of finer algorithms.

6. Abstraction is generally only partially ordered

Independent local abstractions generate incomparable descriptions. A theory that represents granularity by one scalar level discards real structure in the space of computational descriptions.

7. Modal realization projects to occurrent realization

Whenever a modal mechanism is actually run, the realized episode should instantiate an occurrent member compatible with that mechanism. The strong notion therefore entails a corresponding weak actual-run notion.

8. Occurrent realization underdetermines modal realization

Even a complete actual provenance structure need not determine what would have happened in unvisited cases. One occurrent spectrum can therefore be compatible with many modal completions.

9. Local substitution is controlled by interface

If a context can observe only an abstract interface of a subsystem, implementations that are equivalent at that interface may be substituted without changing the coarser surrounding computation. If the context observes additional distinctions, the substitution may cease to be valid.

10. “Same computation” is naturally many-valued in structure, not vague in truth

Two systems may share one computation, many computations, or an entire common subspectrum while differing elsewhere. The richness of shared structure replaces a single primitive yes/no relation.

11. Occurrent repetition can reveal partial laws without becoming modal

If the same component is actually encountered on several input cases, the occurrent structure can accumulate a partial operation table. This can strongly constrain compatible modal mechanisms while leaving unvisited cases genuinely open. More observation can therefore enrich occurrent computation without erasing the occurrent/modal distinction.

12. Resource restriction can enlarge the inevitable common core

Fix an abstract computation and consider all of its admissible realizations satisfying a resource budget. Intersect their spectra. If the budget is tightened, the class of admissible realizations shrinks, so this inevitable common core can only stay the same or grow.

This does *not* imply that stronger compression always recovers the architecture of some chosen original system. Different compact algorithms may still exist. It instead defines a clean notion

of **computational rigidity**: which structures become unavoidable under resource pressure.

6 A compact reconstruction of the theory

The formal theory can be reconstructed from this note in the following order.

1. **Represent actual processes richly.** Use events and value-bearing tokens with actual transport, consumption, production, ordering, control, reuse, and composition. Do not begin from atomic state sequences.
2. **Fix coherent value representation.** Allow heterogeneous local encodings, but require stable typed decoding rules and explicit translations. Do not let the decoder depend on the completed trajectory.
3. **Separate dependence relations.** Use participation/provenance as the base occurrent relation. Distinguish it from actual contribution and from general modal dependence.
4. **Add value support only as an explicit enrichment.** When local operation semantics are supplied, actual values can be organized into minimal sufficient support sets. Record that this is occurrence-indexed but locally modal, not primitive provenance.
5. **Define admissible abstractions.** An abstraction may forget internal distinctions only when the retained computational organization is genuinely supported by the physical organization and representation scheme.
6. **Order abstractions by refinement.** Allow different subsystems to be abstracted independently, giving a poset and, where justified, lattice-like structure.
7. **Build the occurrent spectrum.** For a physical episode, collect all admissible abstractions of its actual valued computational provenance structure. Repeated actual cases may determine partial operation tables.
8. **Build the modal spectrum.** For a mechanism, collect all admissible abstractions whose transition organization and dependence structure remain valid across the relevant alternative inputs and interventions.
9. **Relate the two.** Modal structures project to actual runs; actual runs and their partial laws constrain but generally do not uniquely determine modal completions.
10. **Define implementation by membership.** A computation is implemented when it occurs in the relevant spectrum, not because an observer has privileged a level.
11. **Compare systems spectrally.** Use refinement, containment, spectral equivalence, and common computational core rather than one undifferentiated relation of sameness.
12. **Keep naming separate.** Artifact labels such as program names may require additional historical or conventional criteria beyond computational structure.
13. **Optionally add resource constraints.** Study how the common core of all realizations changes as memory, description length, time, circuit size, locality, or other resources are restricted.

The final picture

The theory has two primary axes and one optional third axis.

Abstraction axis: which distinctions in the computation are retained? This produces a branching refinement structure rather than one scalar level.

Modal axis: are we describing only represented value flow, participation, and provenance actually instantiated; enriching that occurrence with local support information; or describing the full transition and dependence mechanism across unrealized alternatives?

Resource axis: among all realizations of the same abstract computation, which internal structures become unavoidable when the admissible resource region contracts?

The resulting object is not “the computation of the system” but a structured computational spectrum. Objective implementation becomes spectrum membership; similarity between systems becomes structured overlap; coherent representation and provenance block arbitrary trajectory relabeling at the occurrent level; and the occurrent/modal distinction separates what actually happened from the stronger mechanism across unrealized alternatives.

7 From computational identity to computational functionalism

The preceding sections develop an ontology of computational realization in terms of actual episodes, modal mechanisms, and structured spectra of abstraction. That ontology provides the basis for a distinct question:

What structural constraints should any consciousness-making computational property satisfy?

The aim is to constrain the architecture of computational-functionalist theories; the substantive theory supplies the neural or algorithmic property that bears consciousness.

7.1 The primary bearer should be an episode, not an abstract program

An abstract algorithm is a type. It can be instantiated zero, one, or many times. A conscious experience, by contrast, is naturally treated as an occurrence: something happening in a particular system over a particular interval.

A computational functionalism should therefore assign phenomenal properties primarily to **actually instantiated computational episodes**, or to equivalence classes of such episodes under consciousness-irrelevant redescription. The modal computational spectrum remains important for what the system can do, but it need not be constitutive of what is experienced in one actual episode.

Phenomenal Actuality

The phenomenal character of an episode should supervene on properties of the computational process actually instantiated in that episode. No phenomenal difference should arise from a difference that exists only outside the actual process, unless the theory explicitly argues that the modal difference itself is consciousness-relevant.

7.2 Counterfactual differences should not matter by themselves

Suppose every actually encountered instance of a two-input OR gate has input `true,true` and output `true`. Replace those gates by AND gates. On the episode that actually occurs, the same tokens arrive and the same output tokens are generated in the same larger provenance pattern. The two mechanisms differ sharply on `true,false`, but that case never occurs.

A theory that changes the present phenomenology merely because the replacement gate *would have* behaved differently on an unencountered input is making current experience depend on an unrealized branch. That may be a coherent metaphysical position, but it carries a substantial and unnecessary commitment.

Counterfactual Irrelevance

Differences confined to unrealized computational alternatives should not, by themselves, constitute phenomenal differences in the actual episode.

This principle is stronger than merely saying that the system's observable input-output behavior is the same. The actual internal generative process may still matter. What the principle excludes is a phenomenal dependence on differences whose only manifestation lies in unactualized transitions.

OR versus AND: why value support should not automatically be phenomenal

For $\text{OR}(\text{true}, \text{true}) = \text{true}$, the actual output has two alternative singleton minimal supports. For $\text{AND}(\text{true}, \text{true}) = \text{true}$, it has one joint support containing both inputs. Thus the value-support hypergraphs differ even though the actual token flow and generated values can be identical. This is exactly why value support belongs in the theory of computation but should *not* automatically be declared consciousness-constitutive. Its difference here is obtained from local counterfactual semantics. A candidate consciousness theory may use support structure if it has an independent reason to do so, but support difference alone should not be treated as an automatic phenomenal difference.

7.3 Actual generation should matter more than snapshot succession

Counterfactual irrelevance does not imply that any replay of the same sequence of states is equivalent. A movie player can display a sequence of recorded brain states while each frame is produced by reading storage rather than by the preceding computational organization. The state sequence can match while the actual provenance of value production does not.

Generative-Process Sensitivity

A computational functionalism may legitimately depend on how the actual episode is generated: which represented values are produced, transmitted, combined, and reused by which actual events. Equality of snapshots or output traces alone is too weak.

This gives a middle position. It is richer than pure input-output functionalism, but less modal than requiring the entire counterfactual machine to be preserved.

7.4 Representation, substrate, and inert structure should be irrelevant

Several transformations should be presumptively consciousness-neutral when they leave the relevant actual computation unchanged.

Representation Invariance

A coherent renaming or recoding of computational values should not change phenomenology merely because a different physical or symbolic code is used.

Substrate Invariance

Changing the physical material should not change phenomenology when the consciousness-relevant actual computational organization is preserved. Otherwise the theory is no longer computationally functionalist in the intended sense.

Inert-Extension Invariance

Adding dormant branches, disconnected backup hardware, unused lookup tables, or other structures that do not participate in the actual consciousness-relevant process should not alter the phenomenology of that episode.

The last principle is especially important. Without it, attaching a physically inert device whose only role is to change what the system *could* have done can change present experience without changing anything that actually happens.

7.5 Redescription and harmless implementation detail should not multiply experience

The computational spectrum contains many abstractions of one episode. A neuronal description, a circuit description, and a task-level description can all be true at once. It would be pathological to count each as a new consciousness merely because the same process admits several computational witnesses.

Abstraction Consistency

Consciousness-making properties should be invariant under admissible redescriptions and refinements that add only consciousness-irrelevant detail. Multiple spectral descriptions of one physical realization do not by themselves imply multiple phenomenal subjects or episodes.

This principle does not solve subject individuation when genuinely distinct or overlapping subsystems each instantiate candidate consciousness-bearing structures. That is a separate problem that a complete theory must address.

7.6 Robustness and present phenomenality should be separated

Consider an extreme replica of one minute of brain activity. Every component is replaced by a brittle device engineered only to produce the correct outputs on the inputs it will actually encounter during that minute. The device would fail on nearby unrealized cases, but the actual generative provenance of the target episode is preserved.

An actualist computational functionalism should accept the following distinction:

- the replica and the brain can differ radically in **modal robustness**, future capacity, and counterfactual competence;
- they can nevertheless instantiate the same **actual phenomenal episode** if the consciousness-relevant actual process is genuinely preserved.

Capacity–Occurrence Separation

The modal computational spectrum is the natural home of capacities: what conscious episodes a system could support, how robustly it can continue, and how it would respond to other conditions. The phenomenology of the episode currently instantiated should be grounded primarily in its occurrent computational organization.

This avoids making present phenomenology depend on unused competence while preserving a substantial role for modal computation in the theory of conscious agents.

7.7 A class of admissible computational-functionalist theories

The preceding principles do not identify the consciousness-making invariant. They restrict the form of an acceptable candidate. A candidate property P is structurally well behaved if it satisfies, or explicitly justifies violating, the following expectations:

1. P is a property of an actually instantiated computational organization, not merely of an abstract program considered without a realization.
2. P is invariant under coherent value recoding and irrelevant substrate changes.
3. P does not change merely because unrealized transition rules, dormant branches, or inactive backup structures are altered.
4. P can be sensitive to actual internal generation and provenance, so that a movie playback need not be equivalent to a genuine computation.
5. P is stable across admissible abstraction and refinement when the added or removed distinctions are declared consciousness-irrelevant.
6. If P depends on a locally modal object such as value support, the theory must explain why that local counterfactual information is constitutive rather than merely diagnostic.
7. P distinguishes phenomenal occurrence from modal capacity rather than treating them as the same question.

The proposed default architecture

The least conceptually loaded form of computational functionalism is an **actual-process computational functionalism**: phenomenal facts supervene on invariants of the computation actually generated during an episode, modulo coherent recoding, substrate changes, harmless refinement, and other computationally inert transformations. Modal structure describes the mechanism's capacities and constrains possible episodes, but differences confined to unrealized alternatives do not by themselves alter the phenomenology of the episode that actually occurred.

8 Division of theoretical labor in the combined framework

The computational theory and the functionalist constraints now play different roles.

The computational theory tells us what structures are objectively realized: provenance, representation, abstractions, partial operations, optional support structures, modal mechanisms, spectra, refinement, containment, and common computational cores.

The functionalist framework tells us how a consciousness theory should use those structures without inheriting avoidable pathologies. In particular, it warns against four shortcuts:

- identifying consciousness with a bare input-output function;
- identifying it with a relabelable sequence of states;
- making it automatically sensitive to the entire counterfactual machine;
- treating every computational description of one process as a separate phenomenal realization.

The resulting research question is therefore sharper than “which computation is conscious?” It is:

Which invariants of an actually instantiated computational spectrum are phenomenal, and what additional modal information, if any, has an independent reason to be constitutive?

This leaves the substantive theory of consciousness open while specifying a much narrower and more disciplined target space for such theories.

One-sentence version

Physical systems support structured spectra of computational abstractions grounded in coherent representation and actual provenance; modal structure strengthens this into claims about alternative cases, while a disciplined computational functionalism should place present phenomenology primarily on invariants of the actually generated process and treat unrealized counterfactual differences as non-constitutive unless independently justified.